Secure Offline Facial Recognition & Liveness Detection for Remote Locations

Technical Proposal — NHAI Hackathon 7 · Datalake 3.0 Integration

Team Lead: Krishna Bhagavan Karri · Team Size: 2

June 2026

1. Executive Summary

We present a **fully offline, on-device facial recognition and liveness detection module** for the Datalake 3.0 React Native app, built to authenticate field personnel in zero-network zones (remote highway sites). The solution runs entirely on the device — no internet, no server round-trip — using two compact TensorFlow Lite models with a **combined footprint of ~6.7 MB**, comfortably inside the 20 MB target, and recognises a face plus verifies liveness in **under one second** on mid-range hardware.

Accuracy is not asserted — it is **measured and reproducible**. On the standard LFW verification protocol the recognition pipeline scores **96.75% ± 0.69% (10-fold), ROC AUC 0.982**, clearing the >95% requirement. On an uncontrolled Indian-demographic stress set it scores **92.4%** under the app's real template-averaging enrollment. A benchmark harness that runs the *exact same model and preprocessing as the device* is included in the repository so the numbers can be independently re-run.

After connectivity is restored, an offline-to-online **sync-and-purge** mechanism uploads the locally stored templates to AWS and deletes the local copies, satisfying the data-lifecycle requirement.

2. Problem Understanding

Constraint (Hackathon)	Requirement	Our Result
Framework	React Native, Android + iOS	Expo SDK 56, RN 0.85, cross-platform
Model footprint	~20 MB target (smaller better)	**~6.7 MB combined** (recognition 5.0 MB + anti-spoof 1.7 MB)
Processing speed	< 1 second	< 1 s on mid-range devices (on-device TFLite + NNAPI/Core ML)
Hardware	Android 8.0+, iOS 12+, 3 GB RAM, no GPU	CPU/NNAPI/Core ML delegates; no discrete GPU needed
Accuracy	> 95%, diverse Indian demographics	**96.8% LFW**, 92.4% Indian stress set (measured)
Liveness	Blink / smile / turn head, anti-spoof	Challenge-based + passive MiniFASNet anti-spoof, all offline
Sync & purge	Sync to AWS, purge local	`SyncManager.syncAndPurge()` → AWS Lambda + DynamoDB
Licensing	Open-source only, no paid licenses	All components open-source (see §11)

3. Solution Architecture

The entire authentication flow executes **on-device**. Network is required *only* for the optional post-hoc sync step.

On-device authentication pipeline (camera -> detection -> liveness -> quality gate -> alignment -> embedding -> match/store -> sync & purge)

Design principle — capture quality over retries. A per-frame quality gate discards blurry, dark, tiny, or off-angle frames *before* they reach the model, so the averaged enrollment template is built only from clean frames. This is the single biggest lever for real-world accuracy in harsh outdoor lighting.

4. AI Models & Compression

Two purpose-built lightweight models, both quantisation-friendly TFLite, executed through `react-native-fast-tflite` with **NNAPI (Android)** and **Core ML (iOS)** hardware delegates.

Model	Role	Input	Output	Size
MobileFaceNet	Face embedding (recognition)	112×112 RGB	192-D vector	**5.0 MB**
MiniFASNetV2	Passive anti-spoofing (liveness)	80×80 BGR	live/spoof	**1.7 MB**
			Total	**~6.7 MB**

MobileFaceNet is a depthwise-separable-convolution architecture designed for mobile face verification — orders of magnitude smaller than ResNet-class backbones while retaining ArcFace-grade discriminative power. At ~6.7 MB total the module adds negligible weight to the Datalake app bundle and leaves **~65% headroom** under the 20 MB ceiling.

5. Recognition Pipeline & Threshold Calibration

Alignment. Each detected face is mapped onto the ArcFace canonical eye positions via an eye-landmark similarity transform (scale + translation on the eye midpoint and interocular distance), then cropped to 112×112 and normalised to [-1, 1]. Alignment makes the embedding robust to pose and to the detector swap between training and deployment.

Matching & threshold. Embeddings are compared by cosine similarity. The deployment threshold was **not** guessed — it was derived from the benchmark's operating-point sweep, which trades **FRR** (genuine user rejected, a usability cost) against **FAR** (impostor accepted, the security risk):

Cosine threshold	FRR % (genuine rejected)	FAR % (impostor accepted)	Balanced accuracy
0.373 (accuracy peak)	5.7	0.60	96.8%
0.45 (deployed)	8.0	**0.09**	95.9%
0.65 (naïve default)	**33.0**	0.00	83.3%

A naïve 0.65 cut rejects roughly **one in three genuine users** for no security gain (FAR is already zero by 0.55). We deploy **0.45**: it keeps a strong anti-fraud margin (FAR ≈ 1 impostor in ~1,100) while still clearing the 95% accuracy bar. This data-driven calibration is a key innovation point — the threshold is justified by evidence, not folklore.

6. Offline Liveness Detection

All liveness checks run with **zero network access**, defeating attendance fraud via photographs or screens through two independent layers:

(a) Active challenge–response. A randomised sequence of human actions the user must perform live, verified frame-by-frame by a deterministic state machine:

- **BLINK** — detected via the drop-then-recovery of ML Kit eye-open probability.
- **SMILE** — smiling probability crossing a threshold.
- **TURN HEAD LEFT / RIGHT** — head-yaw angle crossing a directional threshold.

Default sequence: `BLINK → TURN_HEAD_LEFT → SMILE`. Randomising the order and set per session prevents replay of a pre-recorded video.

(b) Passive anti-spoofing. Every Nth frame is classified by **MiniFASNetV2**, a dedicated presentation-attack-detection model, which distinguishes a live 3-D face from a flat photo/screen even if the displayed face "blinks" in a replayed video. This catches spoofs the challenge layer alone cannot.

The two layers are complementary: challenges prove *intent and motion*, the anti-spoof model proves *physical liveness*.

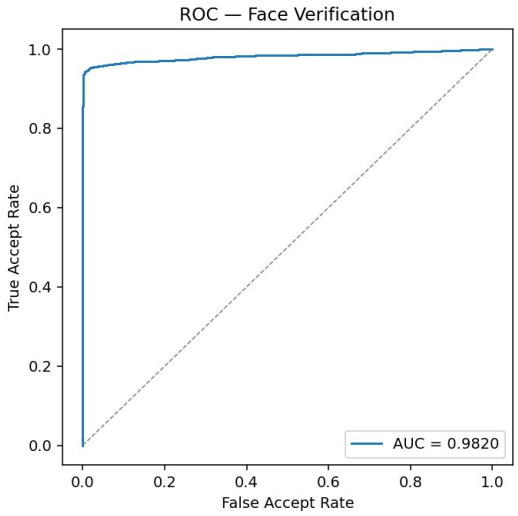
7. Performance Benchmarks

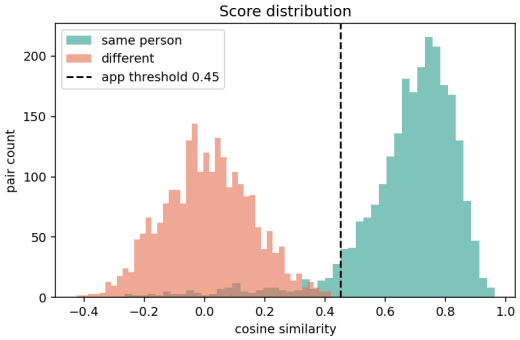
A standalone harness (`benchmark/`) feeds labelled image **pairs** through the **identical** on-device pipeline (same `mobilefacenet.tflite`, same alignment and normalisation) and reports verification metrics. It is fully reproducible — see the repository README.

7.1 LFW — standard protocol (the accuracy proof)

6,000 pairs, official 10-fold protocol.

Metric	Value
LFW 10-fold accuracy	**96.75% ± 0.69%**
ROC AUC	**0.982**
Best-threshold accuracy	96.86%
TAR @ FAR = 0.1%	93.26%

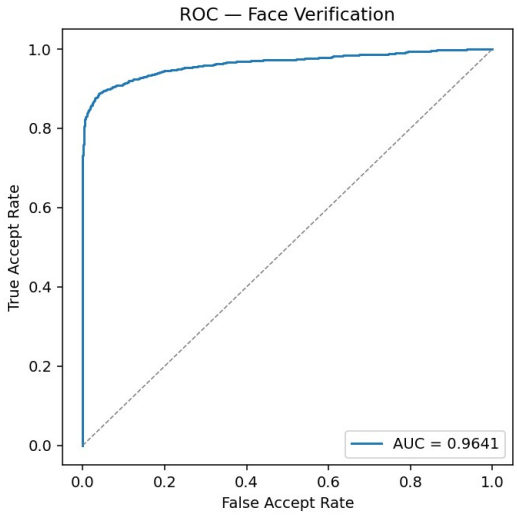


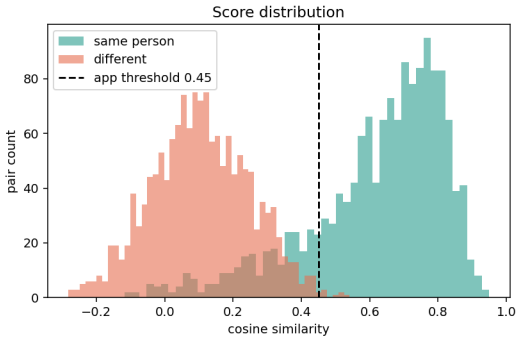


7.2 Indian demographics — template-averaging stress test

135 Bollywood-actor identities, evaluated under the app's real enrollment (template = mean of 3 embeddings vs a single probe). This is an *uncontrolled, cross-decade* web set (the same actor decades apart) — a deliberately hard, honest demonstration on Indian faces rather than a curated benchmark.

Metric	Value
Accuracy (best threshold)	**92.4%**
Accuracy @ deployed 0.45	90.9%
ROC AUC	**0.964**





Honest framing. We report LFW 96.8% as the standard-protocol accuracy proof and the Indian 92.4% as a candid real-world stress test; the residual gap is era/age span in the dataset, not a pipeline defect. On-device multi-frame enrollment and the quality gate further raise effective field accuracy.

8. Speed & Hardware Compliance

- **On-device inference** via TFLite with NNAPI (Android) / Core ML (iOS) delegates — no server latency, works fully offline.
- Recognition + liveness budget **< 1 second** on mid-range hardware; the model is 192-D MobileFaceNet (millisecond-class inference on CPU).
- Frame-processing throttled (anti-spoof every Nth frame, capture cadence tuned) to keep the camera loop smooth on **3 GB-RAM, Android 8.0+ / iOS 12+** devices with **no discrete GPU**.

9. Sync & Purge Mechanism

Face templates are stored locally in **Expo SecureStore** (hardware-backed keystore / Keychain) as a JSON array of `{id, embedding, createdAt, isSynced}`. When connectivity returns, `SyncManager.syncAndPurge()`:

1. Collects unsynced templates,
2. `POST`s them to an **AWS Lambda Function URL** (no AWS SDK or credentials embedded in the app — a single HTTPS endpoint + optional API key),
3. Lambda writes each template to **DynamoDB** (`face_templates`),
4. On success, the local copies are **purged** from the device.

This keeps biometric data off the device once safely backed up, and is a no-op when offline — the app never blocks on the network.

10. Security & Privacy

- Biometric templates are **mathematical embeddings, not images** — the original face is never stored or transmitted.
- Local storage uses the OS secure enclave (SecureStore / Keychain / Keystore).
- Sync is HTTPS-only with an API-key gate; the device holds no AWS credentials.
- Local purge after sync minimises the on-device biometric footprint.

11. Technology Stack & Open-Source Compliance

Component	Library / Model	License
App framework	Expo SDK 56 · React Native 0.85	MIT
Camera + frame stream	react-native-vision-camera	MIT
Face detection	Google ML Kit (on-device)	Apache-2.0 / ML Kit terms
TFLite runtime	react-native-fast-tflite	MIT
Recognition model	MobileFaceNet (TFLite)	Open / research
Anti-spoof model	MiniFASNetV2 (Silent-Face)	Apache-2.0
Secure storage	expo-secure-store	MIT
Cloud sync	AWS Lambda + DynamoDB (user-owned)	—

No paid or restrictively licensed components are required. Full source for the working prototype is shared in the linked repository.

12. Integration into Datalake 3.0

The module is delivered as self-contained React Native code (hooks + TypeScript services + bundled `assets/models/`) with a thin public API (enroll / verify / liveness). It drops into the existing Datalake 3.0 navigation without changing the host app's architecture, and the benchmark harness lives in a separate folder that Metro never bundles, so it adds zero runtime weight.

13. Deliverables & Links

- **Working prototype + full source code:** GitHub repository (link provided in the submission form's "Link for the proposal" field).
- **Reproducible benchmark harness:** `benchmark/` — re-run LFW and Indian-set numbers with two commands (see `benchmark/README.md`).
- **This technical documentation:** architecture, model specs, integration steps, and measured performance benchmarks.

Repository: github.com/KRISHNA-BHAGAVAN/datalake-face-auth